

Desenvolvimento para Dispositivos Móveis

Padrão Repository

Universidade Federal do Rio Grande do Norte

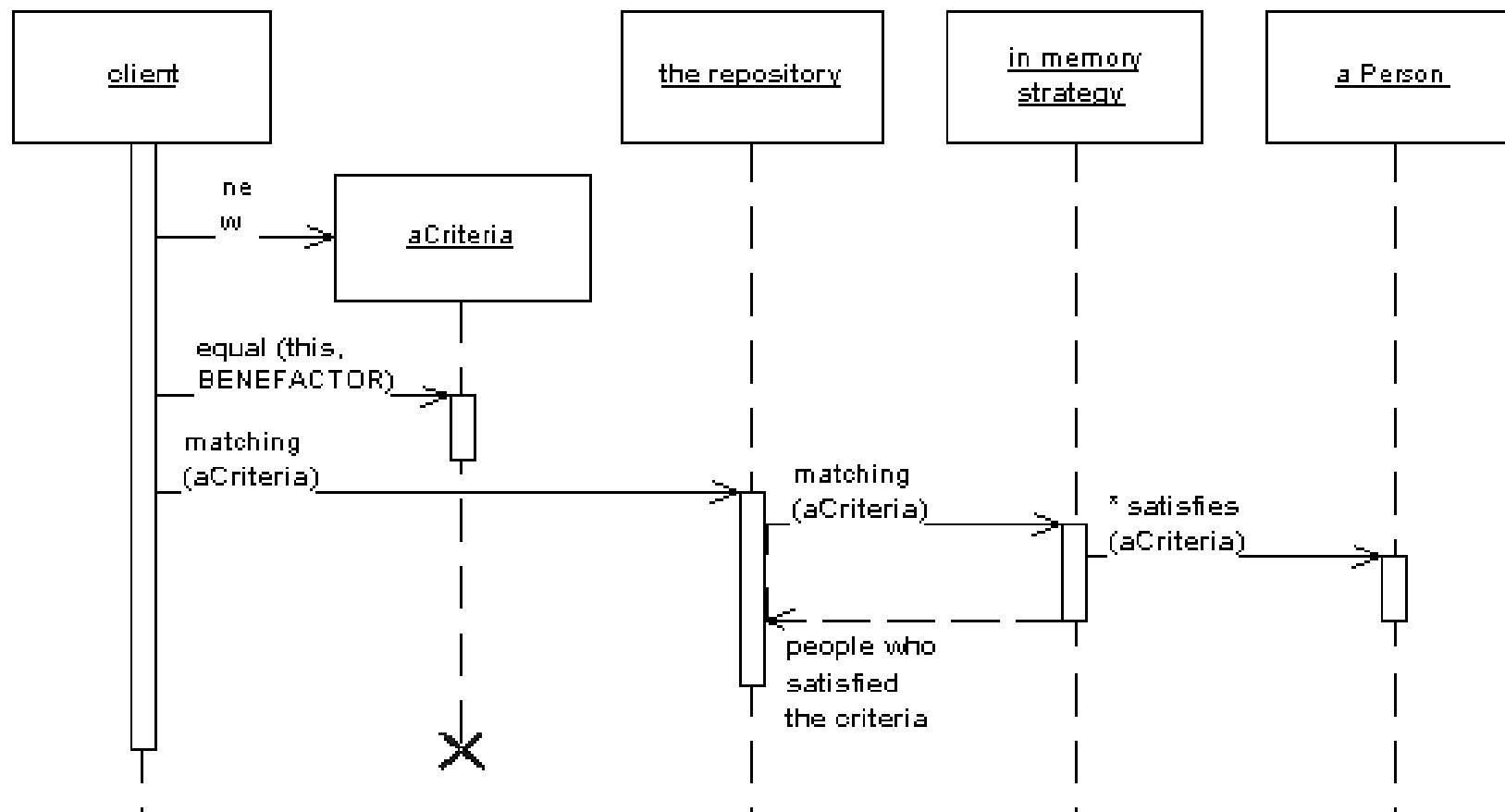
Introdução ao Padrão Repository

- O padrão Repository é um padrão de design de software que oferece uma **abstração sobre a lógica de acesso a dados**, fornecendo uma interface que define **operações específicas para manipular os dados da aplicação**.
- O Repository age como um **intermediário** entre a aplicação e a fonte de dados
 - Banco de dados,
 - serviços web,
 - ou APIs.

Introdução ao Padrão Repository

- O objetivo principal do padrão Repository é desacoplar a lógica de negócios da lógica de persistência de dados, promovendo a separação de responsabilidades.
- Com o uso do Repository, **a aplicação não precisa conhecer os detalhes de como os dados são persistidos ou recuperados.**

Introdução ao Padrão Repository



<https://martinfowler.com/eaCatalog/repository.html>

Motivações para Usar o Padrão Repository

- **Manutenção e Evolução de Código**
 - O Repository centraliza a lógica de acesso a dados, reduzindo a duplicação de código e facilitando a manutenção. Mudanças na forma como os dados são obtidos ou armazenados podem ser realizadas em um único lugar, sem afetar a lógica de negócios.
- **Testabilidade e Mocks**
 - Ao definir uma interface clara para o acesso a dados, o padrão Repository permite a criação de testes unitários mais eficazes. Mocks ou stubs podem ser usados para simular diferentes comportamentos do Repository, permitindo testes focados na lógica de negócios.

Motivações para Usar o Padrão Repository

- **Abstração da Complexidade**

- A complexidade de interagir com diferentes fontes de dados é ocultada do restante da aplicação. A aplicação consome dados através de uma interface simplificada e consistente, sem precisar conhecer detalhes como conexão com o banco de dados ou chamadas de API.

- **Flexibilidade**

- O padrão Repository facilita a mudança de fontes de dados. Por exemplo, pode-se facilmente trocar uma fonte de dados local por uma API remota, ou combinar ambas para otimizar o desempenho, sem impactar o restante da aplicação.

Vantagens do Padrão Repository em Projetos Reais

- Desacoplamento

- O padrão Repository promove o desacoplamento entre a lógica de negócios e a lógica de persistência de dados. Isso significa que a camada de aplicação não precisa estar ciente de como e onde os dados são armazenados, apenas que eles estão disponíveis para serem consumidos.

- Consistência e Reutilização

- Ao utilizar uma interface única para acesso a dados, o Repository garante consistência em como os dados são manipulados em toda a aplicação.

Vantagens do Padrão Repository em Projetos Reais

- Facilidade na Migração e Manutenção
 - Se a aplicação precisar migrar de uma fonte de dados para outra (por exemplo, de um banco de dados SQL para NoSQL), as mudanças são confinadas às implementações dos Repositories, sem impactar as camadas superiores da aplicação.
- Testabilidade Aumentada
 - O uso de uma interface Repository facilita a criação de testes unitários. A lógica de negócios pode ser testada independentemente da lógica de acesso a dados, usando mocks ou stubs para simular o comportamento dos Repositories.

Componentes do Padrão Repository

- **Repository Interface:**
 - Define a interface que especifica as operações de dados, como buscar, salvar, atualizar e deletar entidades. Ela serve como um contrato para as classes que implementam a lógica de acesso a dados.
- **Concrete Repository:**
 - Implementa a interface do Repository, contendo a lógica real de acesso a dados. Essa implementação pode interagir com um banco de dados, uma API, ou qualquer outro tipo de data source. Pode haver múltiplas implementações da mesma interface para diferentes fontes de dados.

Componentes do Padrão Repository

- Entidade/Model

- Representa a estrutura dos dados que serão manipulados. Essas entidades são normalmente objetos simples que contêm atributos correspondentes às colunas de uma tabela no banco de dados ou campos de uma resposta de API.

- Data Source

- É a fonte de dados que o Repository utiliza para buscar ou persistir informações. Podem ser APIs, bancos de dados, caches locais, entre outros. Um Repository pode usar múltiplas fontes de dados e implementar lógica de fallback caso uma fonte não esteja disponível.

Considerações ao Usar o Padrão Repository

- Em aplicações pequenas ou simples, a introdução de Repositories pode adicionar complexidade desnecessária. Quando não há uma separação clara entre as camadas da aplicação, o uso de um Repository pode ser excessivo.
- Apesar de facilitar a manutenção em projetos grandes, a introdução do padrão Repository em projetos menores pode aumentar o custo de manutenção, já que mais classes e interfaces precisarão ser gerenciadas.

Considerações ao Usar o Padrão Repository

- À medida que mais camadas são adicionadas para separar as preocupações, o padrão Repository pode introduzir um overhead de performance, especialmente se não for bem implementado.
- Cada camada adicional pode resultar em mais chamadas de métodos e potenciais atrasos.

Exemplo Prático em Flutter

- Considere um aplicativo Flutter que exibe uma lista de filmes. Esse aplicativo precisa buscar dados de uma API remota e armazená-los em um banco de dados local para cache e acesso offline. A aplicação também deve ser capaz de buscar um filme específico pelo seu ID.

Definição da classe Movie

```
class Movie {  
    final int id;  
    final String title;  
    final String description;  
    final List<String> genre;  
  
    Movie({required this.id, required this.title, required this.description,  
    required this.genre});  
}
```

Repository Interface

```
abstract class MovieRepository {  
    Future<List<Movie>> getMovies({ required int page, required int limit});  
}
```

Repository Interface

- A interface MovieRepository define as operações básicas que a aplicação precisa para manipular os dados dos filmes. Isso inclui o método para obter a lista de filmes.
- Ao definir uma interface, a aplicação pode depender de uma abstração, em vez de uma implementação concreta. Isso promove a flexibilidade e a capacidade de alterar a implementação sem afetar o restante da aplicação.

Repository Implementation

```
class MovieRepositoryImpl implements MovieRepository {  
    final MovieApi api;  
    final MovieDatabase db;  
  
    MovieRepositoryImpl({required this.api, required this.db});  
  
    @override  
    Future<List<Movie>> getMovies() async {  
        try {  
            final movies = await api.fetchMovies();  
            await db.cacheMovies(movies);  
            return movies;  
        } catch (e) {  
            return db.getCachedMovies();  
        }  
    }  
}
```

Pacotes utilizados na aplicação

- freezed_annotation
- freezed
- Dio
- json_annotation
- json_serializable
- Provider
- infinite_scroll_pagination
- Sqflite
- Path
- path_provider
- cached_network_image

Referências

- <https://martinfowler.com/eaCatalog/repository.html>